

Project Proposal – Knowledge-Based Systems

Vehicle Fault Diagnostics System

A system that infers probable vehicle faults based on driver-reported symptoms (sounds, vibrations, dashboard warnings) and OBD-II data, prioritizes them by severity, and recommends appropriate repairs.

Status: Implemented (updated to reflect the current codebase, June 2026)

1. Team Members

Member	ID	Module
Vladimir Morozkin	SV85/2023	Module 1 – Symptom & OBD-II diagnosis
Danila Travkov	SV86/2023	Module 2 – Fault prioritization & repair recommendation

The knowledge model, domain classes, and shared rule infrastructure are designed jointly to ensure consistency of core concepts (`Vehicle` , `Symptom` , `DTCCode` , `SensorReading` , `FaultCandidate` , `Diagnosis`) across both modules.

2. Problem Description

2.1 Motivation

Modern automobiles contain dozens of electronic control units (ECUs) and hundreds of sensors. When something goes wrong, the average driver faces two immediate problems: they do not know whether the fault is dangerous and requires stopping immediately, and they do not know which type of specialist to contact. Misreading the situation can lead to engine seizure, a traffic accident, or an unnecessarily expensive repair caused by delayed action.

Rule-based expert systems are a natural fit here. They reason over a clearly defined knowledge base and can always explain why a particular conclusion was reached - a property that is especially valuable when the stakes involve personal safety and financial decisions.

2.2 Problem Overview and Existing Solutions

The system targets automated vehicle fault diagnosis: given a structured set of driver-reported symptoms and standardized OBD-II diagnostic data (DTC error codes, live sensor values), produce a ranked list of probable faults with confidence scores, urgency ratings, and actionable repair recommendations.

Review of existing solutions:

Solution	Limitation
OBD-II readers (ELM327, FIXD, BlueDriver)	Read DTC codes and display short descriptions. No multi-level inference, no symptom correlation, no urgency assessment.
Professional diagnostic stations (Bosch ESI, Delphi DS)	Designed for trained mechanics with expensive equipment. Not accessible to ordinary drivers.
Mobile applications (Car Scanner, Torque Pro)	Visualize raw OBD-II data but act purely as display tools - no expert reasoning layer.
Academic research (e.g. IDES – Intelligent Diesel Engine System)	Target specific engine types; not available as general solutions.

Advantages of the proposed solution:

- Multi-level forward-chaining that combines symptoms, DTC codes, and sensor values into a diagnosis.
- A CEP module that detects dangerous trends in sensor time-series (overheating trajectory, voltage oscillations, sporadic misfire patterns) before the fault manifests as a visible symptom.
- Backward chaining for hypothesis verification - "Is the root cause X?"
- Fault prioritization by urgency: CRITICAL / HIGH / MEDIUM / LOW, with concrete repair guidance.
- Adaptability to different vehicle types via a template mechanism that parameterizes sensor thresholds by engine type.

3. Methodology

3.1 System Inputs

3.1.1 Driver-reported symptoms

Category	Examples
Sounds	Knocking on acceleration, brake squeal, droning at speed, whining on startup, hissing
Vibrations	Steering wheel vibration under braking, idle vibration, speed-dependent shaking
Dashboard warnings	Check Engine, ABS, Airbag, Oil Pressure, Coolant Temp, TPMS, Battery

Category	Examples
Subjective observations	Power loss, increased fuel consumption, hard starting, exhaust smoke (white/black/blue)
Context of occurrence	Always / only when braking / only on acceleration / only cold / after extended driving

3.1.2 OBD-II diagnostic data

Data type	Details
DTC codes	P (powertrain), B (body), C (chassis), U (network) - active, pending, or historic
Live sensor values	Coolant temp, battery voltage, fuel pressure, O2 sensor, MAF, MAP, RPM, vehicle speed, IAT
Vehicle metadata	vehicleId, engine type (petrol/diesel/hybrid), model year, odometer, engine running flag

Note: Freeze frame and readiness monitors are listed as possible future inputs but are not modelled in the current implementation.

3.2 System Outputs

- **Diagnosis list:** Ranked probable faults with confidence scores
- **Urgency rating per fault:** CRITICAL (vehicle must not be driven), HIGH (repair within 48h), MEDIUM (repair before next service), LOW (monitor, not urgent)
- **Recommended actions:** Service type (auto-electrician, mechanic, tire shop), estimated cost range, what not to do until repaired
- **Rule trace:** The complete chain of fired rules that led to each conclusion
- **Secondary fault warnings:** Faults that may develop if the primary fault is not addressed
- **CEP alerts:** Time-series trend warnings (CepAlert) for overheating, voltage oscillation, misfire patterns, etc.

3.3 Knowledge Base

The knowledge base is organized into three layers corresponding to the three levels of forward-chaining inference.

Layer	Content
Layer 1 – Primary facts	Instances of: Vehicle , Symptom , DTCCode ,

Layer	Content
	SensorReading , VehicleThresholds
Layer 2 – Derived diagnostic facts	SystemProblem , FaultCandidate , correlated symptom groups
Layer 3 – Diagnosis & recommendation	Final Diagnosis , urgency classification, Recommendation , RuleTrace

3.4 Technology Stack

Component	Technology
Language	Java 17
Rule engine	Drools / KIE
Build	Maven multi-module (model , kjar , service)
Backend	Spring Boot REST API
Frontend	React + TypeScript + Vite
Tests	JUnit 5 (DiagnosticRulesTest , CepRulesTest , BackwardChainingTest , SensorThresholdTemplateTest)

3.5 Project Structure

```

SBnZ/
├─ model/      # Domain classes (Vehicle, Symptom, DTCCode, ...)
├─ kjar/       # Drools rules (.drl), templates (.drt)
├─ service/    # Spring Boot REST application
├─ frontend/   # React demo client
└─ docs/      # Class diagram

```

4. Example Rules in Human-Readable Form

#	System	IF (conditions)	THEN (conclusion)	Urgency
1	Engine / Lubrication	Oil pressure warning active AND oil pressure below minimum AND engine running	Probable cause: oil pump failure or severe oil loss. Stop the engine immediately.	CRITICAL
2	Cooling	Coolant temperature above threshold AND coolant warning active	Cooling system problem identified. Schedule urgent repair.	HIGH

#	System	IF (conditions)	THEN (conclusion)	Urgency
3	Ignition / Fuel	DTC P0301–P0304 active AND driver reports power loss	Probable cause: worn spark plug or faulty ignition coil. Repair before next service.	MEDIUM
4	Tyres / TPMS	TPMS warning active AND tyre pressure 10–20% below recommended AND no rapid pressure drop	Probable cause: slow puncture or natural pressure loss. Inflate and monitor.	LOW

These four rules demonstrate the core structure: conditions drawn from sensor readings, DTC codes, and driver-reported symptoms produce a diagnosis with an urgency rating. The full rule base extends this pattern across all major vehicle systems.

5. Rule Base

5.1 Forward Chaining – Three Levels of Inference

All three chaining levels are implemented in `kjar/src/main/resources/rules/vehicle-diagnosis.drl`.

Level 1 – Identify affected system:

```
rule "M1-BR-01 Check engine and P0300 mark ignition system suspect"
when
    Symptom(type == WARNING_CHECK_ENGINE)
    DTCCode(code == "P0300", status == ACTIVE)
then
    insert(new SystemProblem(IGNITION_SYSTEM, SUSPECTED, MEDIUM, ...));
end
```

Level 2 – Identify faulty component:

```
rule "M1-BR-05 EGR restriction candidate from P0401 and black smoke"
when
    Vehicle(engineType == DIESEL)
    DTCCode(code == "P0401", status == ACTIVE)
    Symptom(type == BLACK_EXHAUST_SMOKE)
    SensorReading(name == "maf", value < minMafNormal)
then
    insert(new FaultCandidate(EGR_VALVE, 0.84, HIGH, ...));
end
```

Level 3 – Final diagnosis and recommendation:

```
rule "M2-FWD Final diagnosis and base recommendation"
when
    $candidate : FaultCandidate($component : component, $confidence :
confidence, $urgency : urgency)
    not Diagnosis(component == $component)
then
    insert(new Diagnosis($component, $confidence, $urgency, ...));
    insert(RecommendationFactory.fromDiagnosis(diagnosis));
end
```

Supporting rules:

- M1-FWD Turbo overpressure candidate - creates a turbo candidate when P0234 and boost pressure > 2.0 bar are present (enables M2-BR-07 demo).

5.2 CEP – Sensor Time-Series Trend Detection

OBD-II sensors report values every few seconds. The CEP module aggregates these streams and detects dangerous trends before a fault becomes a visible symptom.

Implementation details:

- Event classes: `SensorEvent`, `DtcEvent` (`@Role(EVENT)`, `@Timestamp`, `@Expires`)
- Result fact: `CepAlert` (replaces the originally planned `CEPEvent` class)
- Rules file: `kjar/src/main/resources/rules/cep/sensor-trend-rules.drl`
- Service: `CepService` uses `ClockTypeOption.get("pseudo")` and `SessionPseudoClock.advanceTime(...)` for demo without real-time waiting
- Separate `KieBase` from the main forward-chaining rules

Rule ID	Name	Window	Trigger	Result
M1-CEP-01 / CEP-01	Overheating trend	5 min	Coolant rise > 15°C	<code>CepAlert(OVERHEATING_TREND)</code>
M1-CEP-02 / CEP-03	Sporadic misfire	10 min	≥ 3 P030x DTC events	<code>CepAlert(SPORADIC_MISFIRE)</code>

Rule ID	Name	Window	Trigger	Result
M1-CEP-03	MAF declining trend	5 min	MAF decline > 4 g/s, final < 10	CepAlert(MAF_DECLINING_TREND)
M2-CEP-01 / CEP-02	Voltage oscillation	2 min	max – min voltage > 1.5 V	CepAlert(VOLTAGE_OSCILLATION)
M2-CEP-02	Rapid tyre pressure drop	3 min	Pressure drop > 15%	CepAlert(TYRE_PRESSURE_DROP)
M2-CEP-03	Repeated critical warnings	10 min	≥ 2 critical DTC events (P0217, P0524, P0562, P0563)	CepAlert(REPEATED_CRITICAL_WARNINGS)

Demo endpoints:

- GET /api/diagnostics/cep/overheating
- GET /api/diagnostics/cep/voltage
- GET /api/diagnostics/cep/misfire
- GET /api/diagnostics/cep/maf
- GET /api/diagnostics/cep/tyre-pressure
- GET /api/diagnostics/cep/critical-warnings

5.3 Template Mechanism – Parameterization by Vehicle Type

Critical sensor thresholds differ between engine types (petrol / diesel / hybrid). A `.drt` template generates concrete DRL rules from a parameter table at runtime.

Files:

- Template: `kjar/src/main/resources/templatetable/sensor_thresholds.drt`
- Service: `ThresholdTemplateService` with `ObjectDataCompiler` → generated DRL → `isolated KieSession`

Parameter table (coolant / oil pressure / charging voltage):

engineType	maxCoolantTemp	minOilPressure	voltageMin	voltageMax
PETROL	110	1.0	13.5	14.8
DIESEL	105	1.2	13.5	14.8
HYBRID	100	0.9	13.8	15.2

Three rule types per engine type generate **9 rules** total:

1. Coolant overheating check
2. Low oil pressure check
3. Battery voltage out of range check

Note: MAF thresholds (`minMafNormal`) are stored in `VehicleThresholds` and used by forward-chaining rules (M1-BR-05), not by the `.drt` template.

Demo endpoints:

- `GET /api/diagnostics/template/drl` - generated DRL text
- `GET /api/diagnostics/template/demo?coolant=108&oil=0.95&voltage=13.6` - comparison across PETROL / DIESEL / HYBRID
- `GET /api/diagnostics/template/scenario?engine=DIESEL&coolant=108&oil=0.95&voltage=13.6` - single engine type

5.4 Backward Chaining – Fault Hypothesis Verification

Module 2 allows the system to verify a directly stated hypothesis ("Is the fault in the lambda sensor?") by recursively checking whether all required conditions are satisfied in working memory.

Files:

- Model: `CausalLink` with `@Position` for recursive query bindings
- Rules: `kjar/src/main/resources/rules/backward/fault-causes.drl`

Queries:

```
query lambdaHypothesis()  
    DTCCode(code == "P0131" || code == "P0132" || code == "P0134", status ==  
    ACTIVE)  
    or (  
        SensorReading(name == "o2_sensor_b1s1", value < 0.1)
```



```

        and Symptom(type == INCREASED_FUEL_CONSUMPTION)
    )
end

query isRootCause(String x, String y)
    CausalLink(x, y;)
    or ( CausalLink(z, y;) and isRootCause(x, z;) )
end

```

Endpoints:

- POST /api/diagnostics/backward/verify-lambda - body: DiagnosticRequest JSON
→ { confirmed, matchingBindings, queryName }
- GET /api/diagnostics/backward/root-cause?effect=BLACK_SMOKE - recursive root causes (e.g. EGR_VALVE , CLOGGED_DPF)

Fault prioritization with accumulate:

```

rule "M2-BR-02 Two high diagnoses in one system escalate combined risk"
when
    $diagnosis : Diagnosis($system : system, urgency == HIGH)
    Number(intValue >= 2) from accumulate(
        Diagnosis(system == $system, urgency == HIGH),
        count(1)
    )
then
    modify($diagnosis) { setUrgency(CRITICAL) };
    insert(new Recommendation(STOP_DRIVING_IMMEDIATELY, ...));
end

```

6. Step-by-Step Reasoning Example

6.1 Input Data (diesel EGR scenario)

Vehicle: Volkswagen Golf 5, 2.0 TDI (diesel), 2007, 187,000 km

Driver-reported symptoms: Check Engine light on, black exhaust smoke under acceleration, noticeable power loss

OBD-II DTC codes: P0401 (EGR Flow Insufficient Detected)

Live sensor values: MAF = 8.2 g/s (expected ≥ 12 for this engine)

Dataset: service/src/main/resources/testdata/diesel-egr.json

6.2 Reasoning Steps

Step	Technique	What happens
1	Template (DIESEL)	Parameter table sets <code>maxCoolantTemp = 105</code> , <code>minMafNormal = 12</code> . MAF <code>8.2 < 12</code> → low MAF condition met.
2	Level 1	P0401 + black smoke → <code>SystemProblem(EGR_SYSTEM, SUSPECTED)</code>
3	Level 2	P0401 + black smoke + low MAF → <code>FaultCandidate(EGR_VALVE, 0.84, HIGH)</code>
4	Level 3	M2-FWD → <code>Diagnosis(EGR_VALVE, 0.84, HIGH)</code> + base recommendation
5	M2-BR-09	EGR fault + black smoke → secondary DPF clogging warning
6	CEP (separate demo)	MAF declining trend over 5 min → <code>CepAlert(MAF_DECLINING_TREND)</code> via <code>GET /cep/maf</code>
7	Backward chaining (separate demo)	Hypothesis "EGR valve fault" verified via causal links → <code>GET /backward/root-cause?effect=BLACK_SMOKE</code>

6.3 Expected Output

- **Primary fault:** Clogged / faulty EGR valve
- **Urgency:** HIGH
- **Recommendation:** Diesel specialist service, EGR valve cleaning or replacement
- **Secondary warning:** DPF clogging risk if EGR fault remains active
- **Rule trace:** Full chain of fired rule names in `DiagnosticReport.ruleTrace`

6.4 Demo Access

What	How
Full forward diagnosis report	<code>GET /api/diagnostics/datasets/diesel-egr/report</code>
Editable JSON evaluation	<code>POST /api/diagnostics/evaluate</code>
CEP MAF trend	<code>GET /api/diagnostics/cep/maf</code>
Backward root cause	<code>GET /api/diagnostics/backward/root-cause?effect=BLACK_SMOKE</code>
UI demo	Start backend + <code>cd frontend && npm run dev</code> → <code>http://localhost:5173</code>

7. Technique Distribution Between Team Members

Technique	Member 1 – Module 1	Member 2 – Module 2
Forward chaining (3+ levels)	Symptoms → system problem → fault candidate	Candidates → diagnosis → priority + recommendation
CEP	Sensor trends, misfire pattern, MAF decline	Voltage oscillation, tyre pressure drop, repeated warnings
Templates (.drt)	Thresholds by engine type	-
Backward chaining	-	Fault hypothesis verification
Accumulate	Aggregated sensor readings (CEP windows)	Urgency score summation (M2-BR-02)

Both members implement forward chaining with a minimum of three chaining levels and use the `accumulate` function. CEP, templates, and backward chaining are each covered by at least one team member, satisfying the requirements for a grade of 9–10.

8. Implementation Architecture

8.1 Runtime Services

Service	Responsibility
<code>DiagnosticService</code>	Forward-chaining evaluation, demo data, backward chaining queries
<code>CepService</code>	CEP KieBase with pseudo clock, trend detection demos
<code>ThresholdTemplateService</code>	<code>.drt</code> compilation and template scenario demos
<code>TestDataService</code>	External JSON datasets (<code>diesel-egr</code> , <code>transmission</code> , <code>tyre-pressure</code>)
<code>DiagnosticController</code>	REST API (<code>/api/diagnostics/*</code>)

Logical modules (Member 1 / Member 2) are separated by rule prefixes (`M1-*` , `M2-*`) and run in a shared Drools session for forward chaining.

8.2 REST API Summary

Endpoint	Description
<code>GET /api/diagnostics/demo</code>	Full demo diagnostic report

Endpoint	Description
GET /api/diagnostics/demo/transmission/report	Transmission scenario (M1-BR-10)
GET /api/diagnostics/rules	Implemented rule catalog
POST /api/diagnostics/evaluate	Evaluate custom DiagnosticRequest JSON
GET /api/diagnostics/datasets	List external JSON datasets
GET /api/diagnostics/datasets/{name}/report	Evaluate dataset and return report
GET /api/diagnostics/template/*	Template mechanism demos
GET /api/diagnostics/cep/*	CEP trend detection demos
POST /api/diagnostics/backward/verify-lambda	Lambda sensor hypothesis verification
GET /api/diagnostics/backward/root-cause	Recursive root cause lookup

8.3 Client Application

React + TypeScript + Vite single-page app in `frontend/`. Proxies `/api` to `http://localhost:8080`. Covers forward reports, datasets, editable JSON evaluation, templates, all six CEP demos, backward chaining, and the rules catalog.

9. Rule Base and Class Diagrams

9.1 Basic Rules

Member 1 – Module 1: Symptom and OBD-II Diagnosis

ID	Area	IF	THEN
M1-BR-01	Engine / DTC	Check Engine active AND DTC P0300 present	Create random misfire symptom group; mark Ignition System as suspect
M1-BR-02	Ignition	DTC P0301–P0304 present AND power loss reported	Create cylinder-specific misfire candidate with MEDIUM urgency
M1-BR-03	Cooling	Coolant temperature above threshold AND coolant warning active	Create Cooling System problem with HIGH urgency
M1-BR-04	Lubrication	Oil pressure warning active AND oil pressure below minimum	Create Oil Pressure fault with CRITICAL urgency

ID	Area	IF	THEN
M1-BR-05	Fuel / Air	DTC P0401 present AND black exhaust smoke AND MAF below expected range (diesel)	Mark EGR flow problem
M1-BR-06	Exhaust / O2	DTC P0131 or P0132 present AND fuel consumption increased	Create lambda sensor fault candidate
M1-BR-07	Electrical	Battery warning active AND voltage below 12.0 V while engine running	Create charging system problem candidate
M1-BR-08	Braking	Brake squeal reported AND vibration while braking	Create brake pad or warped disc candidate with HIGH urgency
M1-BR-09	Tyres / TPMS	TPMS warning active AND tyre pressure 10–20% below recommended	Create low tyre pressure candidate with LOW urgency
M1-BR-10	Transmission	Jerking during gear change AND no engine misfire DTC present	Create transmission or clutch diagnostic candidate

Member 2 – Module 2: Fault Prioritization and Repair Recommendation

ID	Area	IF	THEN
M2-BR-01	Prioritization	Diagnosis has CRITICAL urgency	Recommend STOP_DRIVING_IMMEDIATELY
M2-BR-02	Prioritization	Two HIGH urgency diagnoses affect the same vehicle system	Escalate combined risk level to CRITICAL
M2-BR-03	Repair routing	Fault system is ELECTRICAL_SYSTEM	Recommend auto-electrician service
M2-BR-04	Repair routing	Fault system is BRAKE_SYSTEM	Recommend immediate mechanic / brake service inspection
M2-BR-05	Repair routing	Fault system is TYRE_SYSTEM AND pressure loss is slow	Recommend tyre shop inspection and pressure monitoring
M2-BR-06	Cost estimate	Candidate component is lambda sensor	Attach estimated repair cost range LOW_TO_MEDIUM

ID	Area	IF	THEN
M2-BR-07	Cost estimate	Candidate component is turbo actuator	Attach estimated repair cost range HIGH
M2-BR-08	Driver warning	Exhaust smoke is white AND overheating detected	Warn about head gasket or coolant leak risk
M2-BR-09	Secondary fault	EGR fault remains active AND black smoke present	Add secondary warning for DPF clogging risk
M2-BR-10	Recommendation	Confidence below 0.60 AND urgency is not CRITICAL	Recommend monitoring and additional diagnostic scan

Supporting rules: M1-FWD (turbo overpressure candidate), M2-FWD (final diagnosis and base recommendation).

9.2 CEP Rules

Member 1 CEP Rules

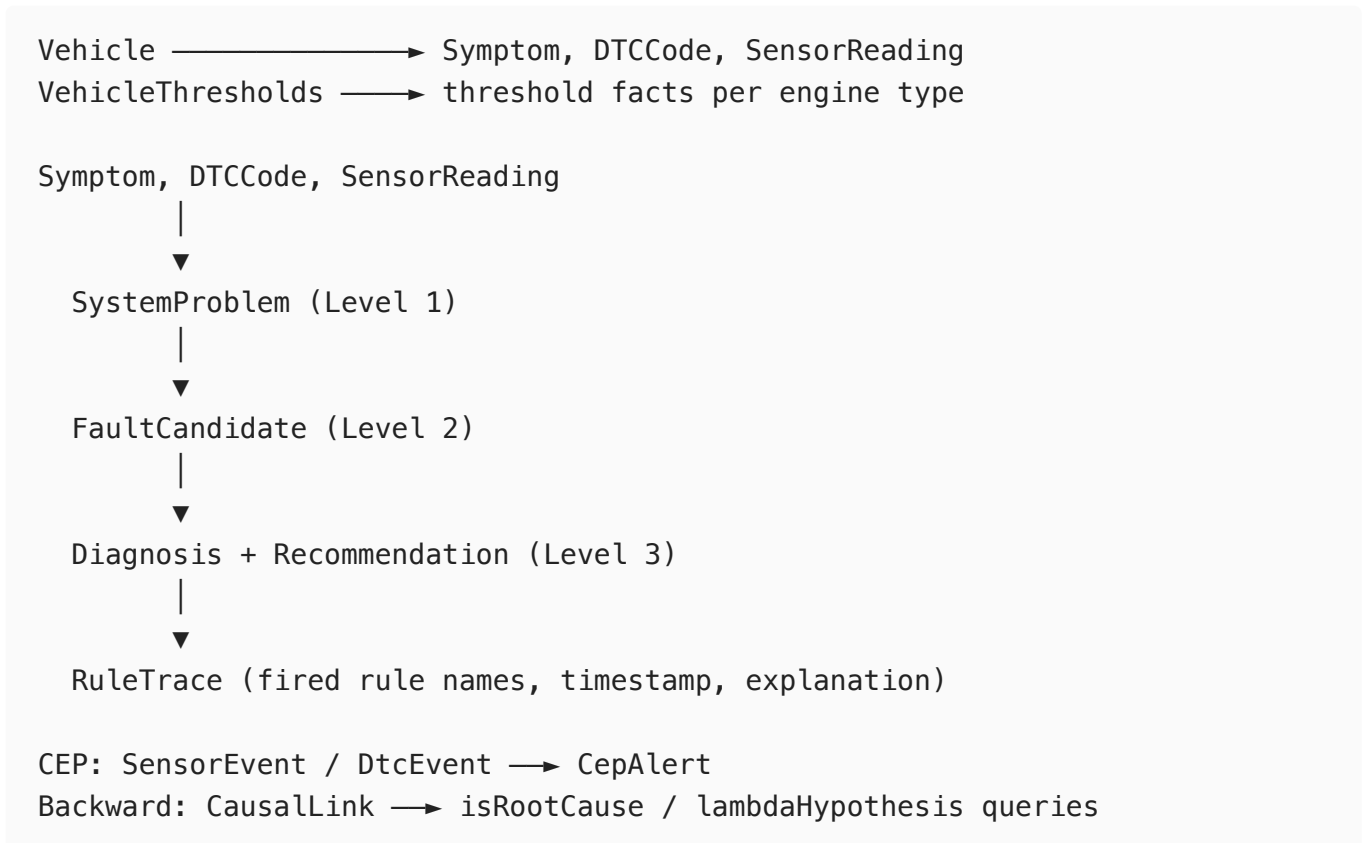
ID	CEP rule	Event pattern	Result
M1-CEP-01	Coolant overheating trend	SensorEvent(coolant_temp) over 5m, max – min > 15°C	CepAlert(OVERHEATING_TREND)
M1-CEP-02	Sporadic misfire pattern	≥ 3 P030x DTC events in 10m	CepAlert(SPORADIC_MISFIRE)
M1-CEP-03	MAF declining trend	MAF readings decline > 4 g/s over 5m, final < 10	CepAlert(MAF_DECLINING_TREND)

Member 2 CEP Rules

ID	CEP rule	Event pattern	Result
M2-CEP-01	Battery voltage oscillation	Voltage min/max difference > 1.5 V over 2m	CepAlert(VOLTAGE_OSCILLATION)

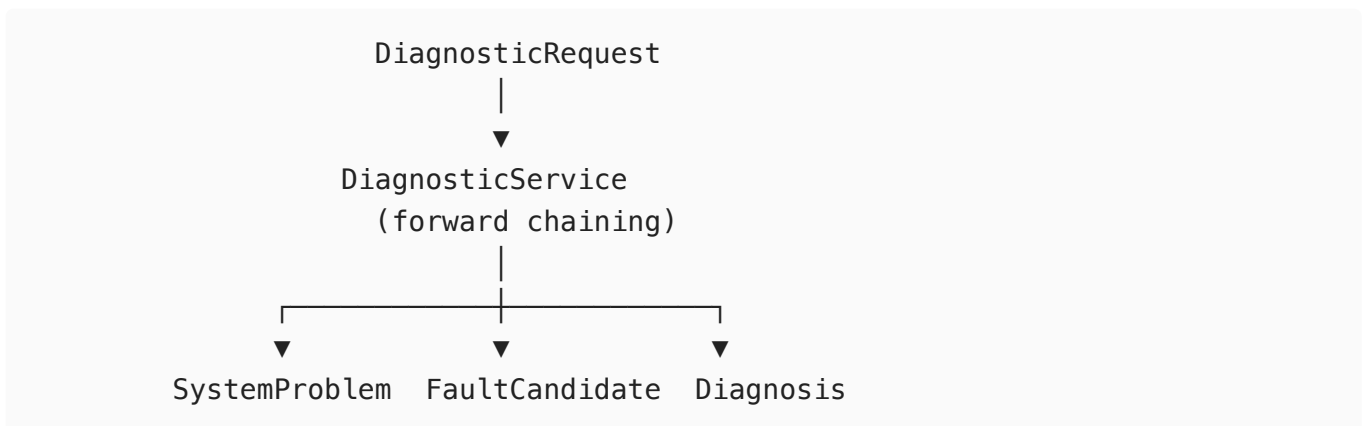
ID	CEP rule	Event pattern	Result
M2-CEP-02	Rapid tyre pressure drop	TPMS pressure drops > 15% in 3m	CepAlert(TYRE_PRESSURE_DROP)
M2-CEP-03	Repeated critical warnings	≥ 2 critical warning DTC events within 10m	CepAlert(REPEATED_CRITICAL_WARNINGS)

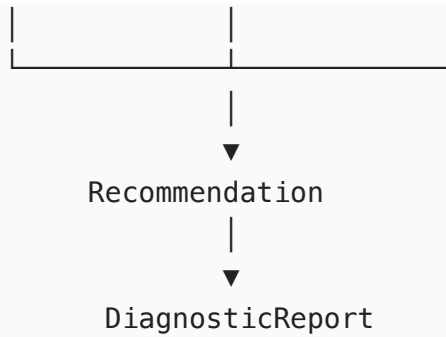
9.3 Domain Class Diagram



Full Mermaid diagram: [SBnZ/docs/class-diagram.md](#)

9.4 Module Responsibility Diagram





CepService → CepAlert (separate KieBase, pseudo clock)
ThresholdTemplateService → generated DRL from .drt
Backward queries → lambdaHypothesis, isRootCause

10. Conclusion

The vehicle fault diagnostics system delivers multi-layer expert reasoning that combines driver-reported symptoms, OBD-II data, and sensor time-series trends into actionable fault diagnoses. It is designed to be understandable to a non-technical driver while providing transparent, traceable reasoning for every conclusion.

By implementing all three advanced techniques required by the course (CEP, templates, backward chaining) alongside forward chaining with three or more levels of inference, the system reaches a diagnostic capability that substantially exceeds existing commercial OBD readers.

The implementation is complete and demonstrable through a Spring Boot REST API, a React client, external JSON datasets, and automated tests. This document reflects the actual state of the codebase as of June 2026.